

12 comandos GitHub CLI: Descripción, Función, Sintaxis y Ejemplo

En este documento se mostrará por cada comando en qué consiste, qué hace, la forma correcta de escribirlo en la línea de comandos y su correcto funcionamiento utilizado ya en un repositorio.

Se tiene una tabla donde se mostrará una descripción breve de cada comando elegido:

Descripción general de su función

Número	Comando	Descripción
1	<code>gh alias delete</code>	Elimina alias configurados previamente.
2	<code>gh copilot</code>	Ejecuta la interfaz de línea de comandos de GitHub Copilot.
3	<code>gh secret</code>	Administra secretos a nivel de repositorio u organización para uso en GitHub Actions, Dependabot o Codespaces.
4	<code>gh exit-codes</code>	Muestra las convenciones estándar que sigue <code>gh</code> con respecto a los códigos de salida (exit codes).
5	<code>gh reference</code>	Es un comando de ayuda global que despliega el manual completo y detallado listando absolutamente todos los comandos, subcomandos y parámetros disponibles.

Número	Comando	Descripción
6	<code>gh browse</code>	Transiciona de la terminal al navegador web para ver e interactuar con: Issues, Pull requests, contenido y configuración del repositorio.
7	<code>gh pr</code>	Administra y trabaja con los Pull Requests de GitHub.
8	<code>gh repo</code>	Administra y trabaja con repositorios de GitHub.
9	<code>gh cache delete</code>	Elimina cachés de GitHub Actions. Su eliminación requiere autorización previa con el alcance <code>repo</code> .
10	<code>gh alias</code>	Crea atajos personalizados para comandos de <code>gh</code> o permite agrupar múltiples comandos bajo un solo nombre.
11	<code>gh api</code>	Realiza una solicitud HTTP autenticada a la API de GitHub y muestra la respuesta. Admite endpoints v3 o acceso a GraphQL v4.
12	<code>gh search</code>	Realiza búsquedas de forma global a través de todo GitHub (repositorios, código, usuarios, etc.).

Utilidad de cada comando y sintaxis

A continuación se detalla el propósito de cada comando dentro del flujo de trabajo de desarrollo y su estructura sintáctica base.

1. **gh alias delete** * **Utilidad:** Útil para mantener limpia la configuración de tu terminal, eliminando atajos de comandos obsoletos o creados por error. * **Sintaxis:** `gh alias delete <nombre-del-alias> [flags]`
2. **gh copilot** * **Utilidad:** Permite interactuar con la inteligencia artificial de Copilot sin salir de la terminal, ideal para solicitar sugerencias de comandos complejos en Bash, configuraciones de Git o scripts. * **Sintaxis:** `gh copilot <command> [flags]`
3. **gh secret** * **Utilidad:** Esencial para salvaguardar credenciales corporativas, tokens o claves de bases de datos que los flujos automatizados de CI/CD requieren sin exponerlos en el código fuente. * **Sintaxis:** `gh secret <command> [flags]`
4. **gh exit-codes** * **Utilidad:** Sirve como referencia técnica para la creación de scripts en Bash o Python. Permite identificar si un comando CLI se ejecutó con éxito (código 0) o si falló, facilitando el manejo de errores. * **Sintaxis:** `gh help exit-codes`
5. **gh reference** * **Utilidad:** Funciona como un manual offline exhaustivo (similar a las páginas `man` en Linux), ideal para consultar opciones avanzadas de la herramienta sin depender de un navegador de internet. * **Sintaxis:** `gh reference`
6. **gh browse** * **Utilidad:** Ahorra clics y tiempo de navegación al abrir directamente la página exacta del repositorio remoto en el navegador predeterminado desde la carpeta de trabajo actual. * **Sintaxis:** `gh browse [<number> | <path>] [flags]`
7. **gh pr** * **Utilidad:** Agiliza el flujo de trabajo colaborativo permitiendo crear, revisar, aprobar o fusionar propuestas de cambios directamente desde la terminal de Git Bash. * **Sintaxis:** `gh pr <command> [flags]`
8. **gh repo** * **Utilidad:** Facilita la administración total de los repositorios remotos (crear, clonar, hacer forks o eliminarlos) integrando perfectamente las capacidades web de GitHub al entorno local. * **Sintaxis:** `gh repo <command> [flags]`
9. **gh cache delete** * **Utilidad:** Optimiza el espacio de almacenamiento y previene errores en las pruebas automatizadas al purgar archivos temporales y dependencias oxidadas en GitHub Actions. * **Sintaxis:** `gh cache delete [<cache-id> | <cache-key> | --all] [flags]`
10. **gh alias** * **Utilidad:** Incrementa la productividad del desarrollador al abreviar comandos largos que se usan con frecuencia durante el desarrollo de proyectos. * **Sintaxis:** `gh alias <command> [flags]`
11. **gh api** * **Utilidad:** Otorga flexibilidad absoluta al permitir consultas directas a los servidores de GitHub; útil para obtener datos estructurados (como JSON) que la CLI no extrae por defecto. * **Sintaxis:** `gh api <endpoint> [flags]`

12. `gh search` * **Utilidad:** Actúa como un motor de descubrimiento rápido para localizar fragmentos de código, librerías o proyectos open-source específicos sin interrumpir el ritmo de programación en la terminal. * **Sintaxis:** `gh search <command> [query] [flags]`

Ejemplos de ejecución

1. Eliminar un alias configurado (`gh alias delete`)

```
gh alias delete pv
```

Elimina el atajado de comando que habia sido guardado previamente con el nombre 'pv'.

2. Solicitar asistencia de código(`gh copilot`)

```
gh copilot suggest "Cómo compilar un proyecto de Java en la terminal"
```

Pide a la IA una sugerencia exacta sobre los comandos necesarios para realizar esa acción.

3. Crear un secreto en el repositorio (`gh secret`)

```
gh secret set DB_PASSWORD
```

La terminal solicitará ingresar de manera segura la contraseña de la base de datos para registrarla en el repositorio remoto.

4. Consultar los códigos de salida (`gh exit-codes`)

```
gh help exit-codes
```

Despliega la documentación detallada sobre cómo gh reporta sus éxitos y errores al sistema operativo.

5. Abrir la referencia completa offline (`gh reference`)

```
gh reference
```

Inicia el despliegue paginado del manual de todas las funcionalidades de GitHub CLI.

6. Navegar rápidamente al código remoto (`gh browse`)

```
gh browse settings
```

Abre automáticamente el navegador en la pestaña de "Configuración" del repositorio en el que se encuentra localmente.

7. Crear un Pull Request (`gh pr`)

```
gh pr create --title "Corrección de errores" --body "Se actualizó la lógica"
```

Sube el cambio al repositorio original agregando de forma inmediata un título y un cuerpo explicativo.

8. Clonar un repositorio remoto (gh repo)

```
gh repo clone lolmedo-99/repositorio
```

Descarga una copia idéntica del repositorio especificado directamente a la computadora local.

9. Limpiar caché de flujos de trabajo (gh cache delete)

```
gh cache delete --all
```

Limpiar absolutamente todos los cachés almacenados en las GitHub Actions del repositorio actual.

10. Configurar un nuevo alias (gh alias)

```
gh alias set co checkout
```

Crea el alias 'co', permitiendo escribir gh co en lugar de teclear gh checkout en futuras ocasiones.

11. Consultar la API de GitHub (gh api)

```
gh api repos/Paola1426/repositorio2/issues
```

Realiza una petición HTTP GET para obtener la lista de incidencias (issues) del repositorio de Paola1426 y la devuelve en la consola.

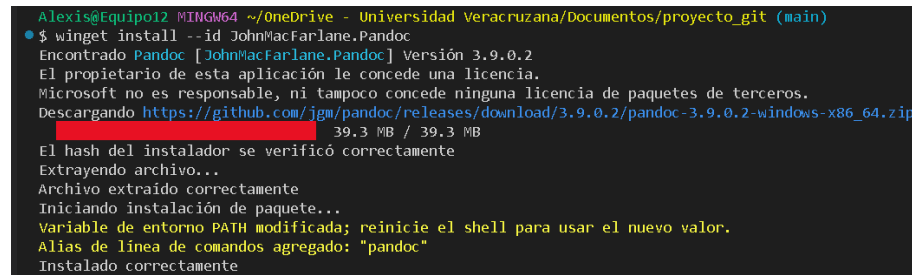
12. Buscar proyectos en GitHub (gh search)

```
gh search repos "digital logic simulator" --language=python
```

Busca en todo GitHub repositorios relacionados con simuladores lógicos digitales que estén programados específicamente en Python.

Evidencia de conversión de Markdown a PDF con Pandoc

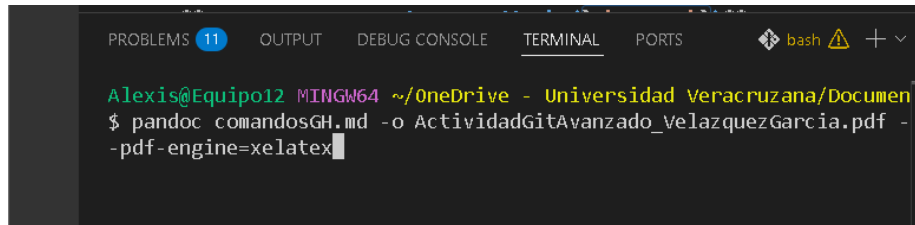
Pandoc descargado desde terminal



```
Alexis@Equipo12 MINGW64 ~/OneDrive - Universidad Veracruzana/Documentos/proyecto_git (main)
$ winget install --id JohnMacFarlane.Pandoc
Encontrado Pandoc [JohnMacFarlane.Pandoc] Versión 3.9.0.2
El propietario de esta aplicación le concede una licencia.
Microsoft no es responsable, ni tampoco concede ninguna licencia de paquetes de terceros.
Descargando https://github.com/jgm/pandoc/releases/download/3.9.0.2/pandoc-3.9.0.2-windows-x86_64.zip
39.3 MB / 39.3 MB
El hash del instalador se verificó correctamente
Extrayendo archivo...
Archivo extraído correctamente
Iniciando instalación de paquete...
variable de entorno PATH modificada; reinicie el shell para usar el nuevo valor.
Alias de línea de comandos agregado: "pandoc"
Instalado correctamente
```

Figure 1: Pandoc

Comando ejecutado para la conversión a PDF

A screenshot of a Git Bash terminal window. The window has a dark background and a light-colored title bar. At the top, there are tabs for 'PROBLEMS' (with a blue circle containing '11'), 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL' (which is active), and 'PORTS'. To the right of the tabs is a 'bash' icon and a yellow warning triangle. The terminal content shows the prompt 'Alexis@Equipo12 MINGW64 ~/OneDrive - Universidad Veracruzana/Documen' followed by the command '\$ pandoc comandosGH.md -o ActividadGitAvanzado_VelazquezGarcia.pdf -pdf-engine=xelatex' with a cursor at the end of the command.

```
Alexis@Equipo12 MINGW64 ~/OneDrive - Universidad Veracruzana/Documen
$ pandoc comandosGH.md -o ActividadGitAvanzado_VelazquezGarcia.pdf -
-pdf-engine=xelatex
```

Figure 2: Ejecución de comando desde Git Bash para la conversión